

CO2-ASSESSMENT TOOL-3

Name: R.GAUTHAM KRISHNA

Reg.no: 192421128

Course name: DESIGN AND ANALYSIS OF ALGORITHMS

Code: CSA0613

Faculty: Dr. F. MARY HARIN FERNANDEZ

Date: 29-07-2026

Question 28

Title

Debugging Pair Sum Code (Duplicate and Mirror Pair Detection)

Problem Statement

The given Python program is intended to find all pairs of elements whose sum equals a given target value. However, the implementation contains a major issue because it compares every element with every other element. As a result, the same pair is detected multiple times (mirror pairs), leading to redundant output.

The objective is to debug the code by eliminating duplicate pair detection, ensuring that each valid pair is checked only once, improving efficiency while preserving correctness.

Original Code

```
1 def pair_sum_all(arr, target):
2     pairs = []
3     for i in range(len(arr)):
4         for j in range(len(arr)):
5             if i != j and arr[i] + arr[j] == target:
6                 pairs.append((i, j))
7     return pairs
```

Problem Analysis

Issue 1: Duplicate Pair Detection

- The nested loops compare every element with every other element.
- The pair (0,1) and (1,0) represent the same solution but are both added.

Issue 2: Redundant Comparisons

- Every pair is checked twice.
- This increases unnecessary computations.

Issue 3: Inefficient Output

- The output contains mirror pairs, making the result difficult to interpret.

Corrected Algorithm

1. Accept the input list and target value.
2. Initialize an empty list to store valid pairs.
3. Traverse the list using the first loop.
4. Start the second loop from $i + 1$ instead of 0.
5. Check whether the sum of the selected pair equals the target.
6. If it matches, store the pair.
7. Return the list of unique pairs.

Python Program (Source Code)

```
1  # Optimized Pair Sum using Brute Force
2
3  def pair_sum_all(arr, target):
4      pairs = []
5
6      # Check each pair only once
7      for i in range(len(arr)):
8          for j in range(i + 1, len(arr)):
9              if arr[i] + arr[j] == target:
10                 pairs.append((i, j))
11
12     return pairs
13
14
15 arr = [2, 4, 3, 5, 7, 8, 9]
16 target = 7
17
18 result = pair_sum_all(arr, target)
19
20 print("Array :", arr)
21 print("Target:", target)
22 print("Pairs :", result)
```

OUTPUT

Array : [2, 4, 3, 5, 7, 8, 9]

Target: 7

Pairs : [(0, 3), (1, 2)]

Explanation

- The corrected program starts the inner loop from $i + 1$.
- This ensures each pair is checked only once.
- Mirror pairs such as (1,0) are never generated after (0,1).
- Duplicate comparisons are eliminated.
- The program still finds every valid pair without missing any solution.
- This approach improves readability and reduces redundant operations.

Time Complexity Analysis

Operation	Complexity
-----------	------------

Nested Loop Pair Search	$O(n^2)$
-------------------------	----------

Overall Time Complexity	$O(n^2)$
-------------------------	----------

Space Complexity	$O(k)$ (where k is the number of valid pairs)
------------------	---

Advantages

- Eliminates duplicate and mirror pairs.
- Reduces redundant comparisons.
- Produces clean and unique output.
- Preserves correctness.
- Easy to understand and maintain.

Conclusion

The original implementation generated duplicate and mirror pairs because every element was compared with every other element. The corrected version starts the inner loop from $i + 1$, ensuring that each pair is checked exactly once. This removes redundant comparisons while maintaining correctness. The optimized algorithm still has a time complexity of $O(n^2)$ but produces cleaner and more efficient results, making it more suitable for practical applications.